AI 动画辅助实现的方案与实践

原创 用户场景营销技术 大淘宝技术 2025年10月13日 03:10 浙江 11人

本文介绍了一种基于AI技术辅助实现微动效的解决方案，旨在提升淘宝秒杀业务中动画开发的效率与还原质量。针对Lottie动画在性能、动态适配和文件体积上的局限，以及DOM/SVG手动实现动画时存在的沟通成本高、实现慢、维护难等问题，作者团队在塔罗平台基础上开发了两个AI动画助手：**Lottie动画开发助手**和**SVG动画开发助手**。这两个工具将非结构化的视觉动画转化为可编程、高性能、易维护的前端实现，显著降低了开发门槛，使原本耗时数小时的动画开发流程缩短至几分钟，实现了从设计到代码的高效落地。

01 前言

本文所述动画助手**仅适用于微动效场景**，旨在提升开发效率，不替代 Lottie 动画本身的使用，且对设计师导出的 Lottie 文件存在一定的规范要求。

02 背景与成果

淘宝秒杀业务存在动画实现需求密集、动效细节丰富、且需求变更频繁的特点，我们主要采用 Lottie 或 DOM 动画来实现各类动效。



动画1



动画2

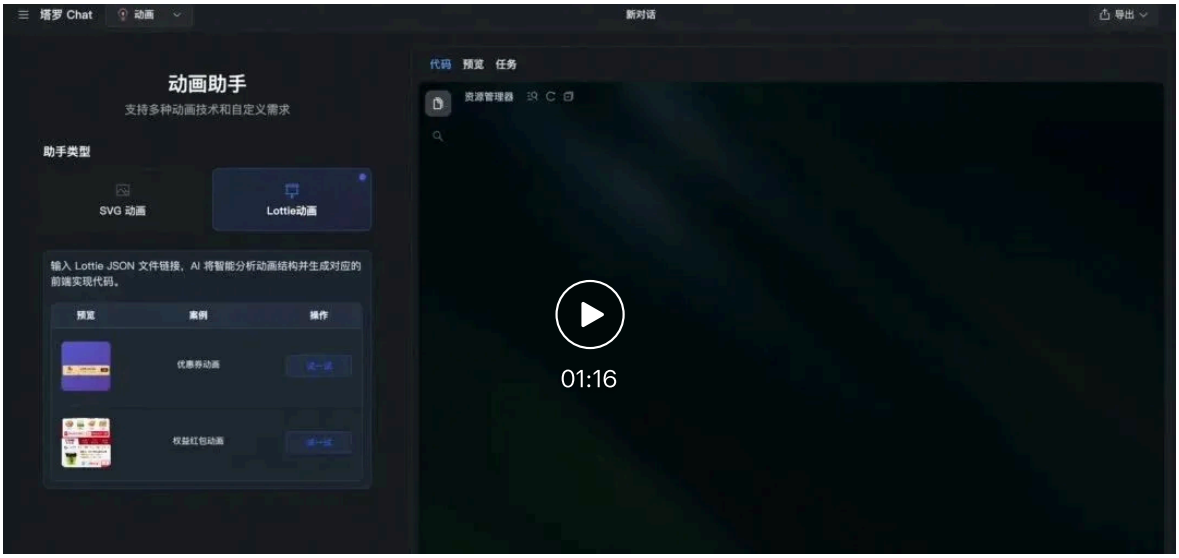


动画3

除了复杂的**纯动画可直接使用 Lottie** 实现外，日常开发中更常见的是上面这些轻量的微动效（动画）—例如卡券、红包等组件在不同状态下的摆动、呼吸、旋转等。**这类动效通常用在 React 组件中，伴随状态变化出现，比如用户点击、倒计时结束、滚动停止或领取成功时触发**。一个组件往往有多个状态，也需要响应不同的交互，每个状态切换都可能需要对应的动画来提示用户。虽然动画本身不复杂，但要和组件状态对齐、控制播放时机、处理中断和重复触发等问题，手动实现起来并不简单，调试和维护成本也高（详见下文实际开发中的典型痛点）。

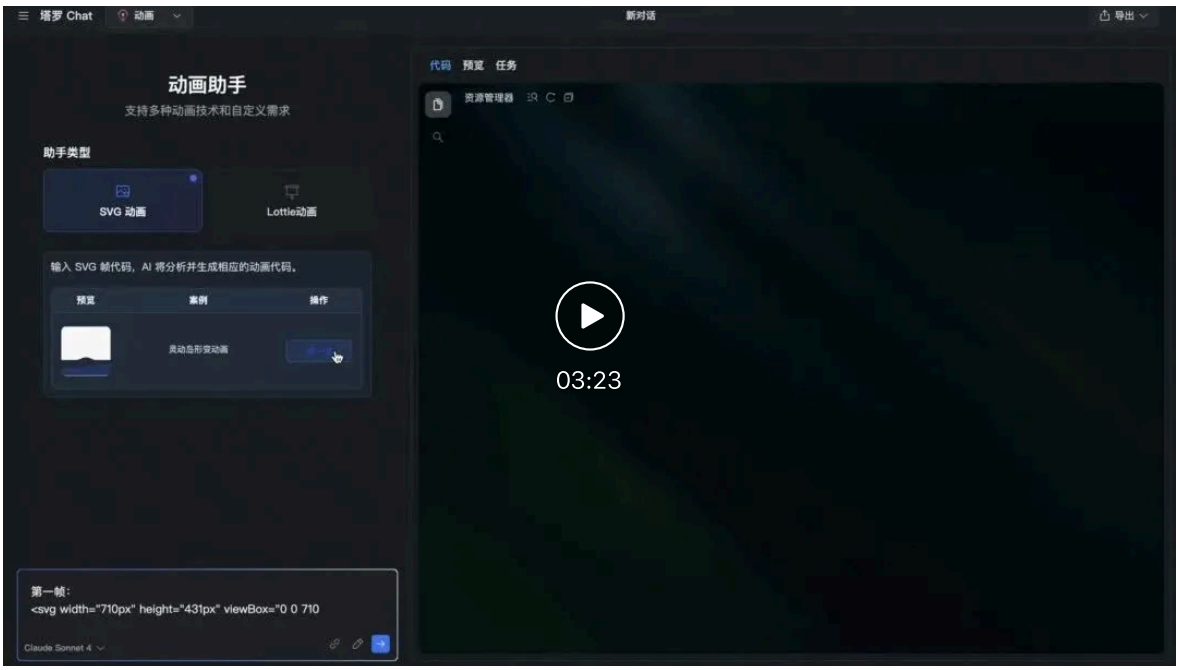
我们做了什么：我们聚焦淘宝秒杀业务中的典型微动效需求，围绕淘宝秒杀业务中的开发痛点，借助 AI 能力设计并实现了两个动画开发助手，显著提升了业务动画需求的开发效率与动画还原质量，分别是：

- **Lottie 动画开发助手**：通过工程的方式对 Lottie 文件进行解析与映射，并用 AI 将其关键帧数据和动画逻辑自动转换为结构化的 DOM 动画实现。该方案在保留 Lottie 动画表现力的同时，提升了动画的加载性能、可维护性与灵活性。





- **SVG 动画开发助手**：基于 AI 技术自动生成并优化 SVG 动画代码，尤其针对路径变形、形变动画等复杂场景，实现关键帧间的平滑过渡，输出轻量、可维护的动画代码，大幅降低开发门槛。



- 以及正在开发中的其他助手。

03 为什么要使用 AI 辅助动画实现

Lottie 的局限性：无法满足动态与性能的双重需求

在基于 Lottie 实现动画的过程中，我们发现其存在以下问题：

- **加载性能问题和失败风险：**Lottie 动画文件可能存在加载缓慢或失败的情况，需要额外的容错和兼容处理。
- **文件体积问题：**对于不规则形变等复杂动画（如第一章的动画示例 3），Lottie 只能通过帧图片实现，导出体积接近 1MB，影响加载效率和动画衔接流畅度。
- **动态效果实现复杂：**对于倒计时、多状态切换等动态内容，需要从 Lottie 动画制作阶段就开始结构化设计——例如预留动态图层、命名规范关键元素，另外仍需开发侧进行大量解析和运行时处理（借助 lottie-web 等库），灵活性远不如原生代码实现。

Lottie 更适合表现“静态播放”的纯视觉动效，也可以作为动效技术栈的“补充手段”，但不是“通用解决方案”，我们更希望有一种**可编程、高性能、易维护**的动画实现方案 —— 而这正是 DOM/SVG 动画的优势所在。

DOM 实现动画的难点

相比较 Lottie 实现动画效果，DOM 实现动画具备高度的灵活性，但在以往基于 DOM 的动画开发模式中，我们遇到三个核心问题 — **沟通难**和**实现慢**以及**成本高**，导致整体效率低下，在复杂动效场景下尤其明显。

- **沟通难**：设计师与前端开发之间存在“信息断层”，设计团队通常通过动效示例（如 Lottie 预览、AE 视频或 GIF）传达动画意图，但这些素材是“黑盒”式的 — 前端开发难以从中准确获取关键参数，例如：
  - 某个元素是第几秒出现的?
  - 缩放动画持续多久？是从 0.8 放大到 1.2，还是直接从 0 到 1?
  - 贝塞尔曲线是缓入缓出，还是有弹跳效果?
  - 多个元素之间的动画是否存在重叠或延迟?

这些信息必须靠开发者“猜”或反复确认，**开发者无法从视频中精确反推时间轴、数值变化和运动模型**。虽然可以借助工具调试，但依然依赖人为判断，效率低且还原度不可控。

- **实现慢：**实现过程依赖“手动调试” + “肉眼比对”，即使大致理解了动画意图，实际编码仍是一个“试错式”的过程：
  - 使用 CSS 或 JavaScript 手动编写动画关键帧；
  - 不断播放设计提供的视频 demo，逐帧比对元素位置、大小、透明度等变化；
  - 反复调整 transform、opacity、timing function 等参数，直到“看起来差不多”；
  - 提交后由设计师验收，但经常因为细微差异被打回修改。

随着动画复杂度上升（如路径动画、形变、蒙版等），时间成本迅速增长。

- **成本高：**对于复杂不规则形变动画，实现门槛极高，设计师提供的形变动画（如动画 3）往往涉及路径变形、形态插值等高级动效，这类动画通过前端手动开发几乎难以精准还原，或实现成本极高。主要原因包括：
  - SVG 的 `path` 数据复杂且可读性差，难以手动调整；
  - 形变动画依赖关键帧路径匹配，开发需逐帧解析并生成对应 `d` 属性；
  - 缺乏可视化编辑支持，调试和维护极为困难；
  - 动画逻辑与结构耦合度高，后续修改成本大。

> 📌 一个典型场景：一个看似简单的“按钮弹出 + 图标旋转 + 文字渐显”组合动画，可能涉及 3 个元素、5 段动画、多个时间轴对齐问题。开发 + 调试 + 验收流程往往耗时 2—4 小时。

我们意识到，动画开发的核心挑战，不在于能否实现某个效果，而在于如何高效、准确地将设计意图转化为可执行、可维护的代码。无论是依赖资源文件的 Lottie，还是灵活但低效的手写 DOM 动画，都难以在性能、准确性、开发效率与维护成本之间取得平衡。

- AI 的价值：做“人做不到的事”

> 如何将非结构化的视觉表达（Lottie）转化为结构化的、可执行的动画程序？

现在我们可以借助 AI 来解决这一难题：

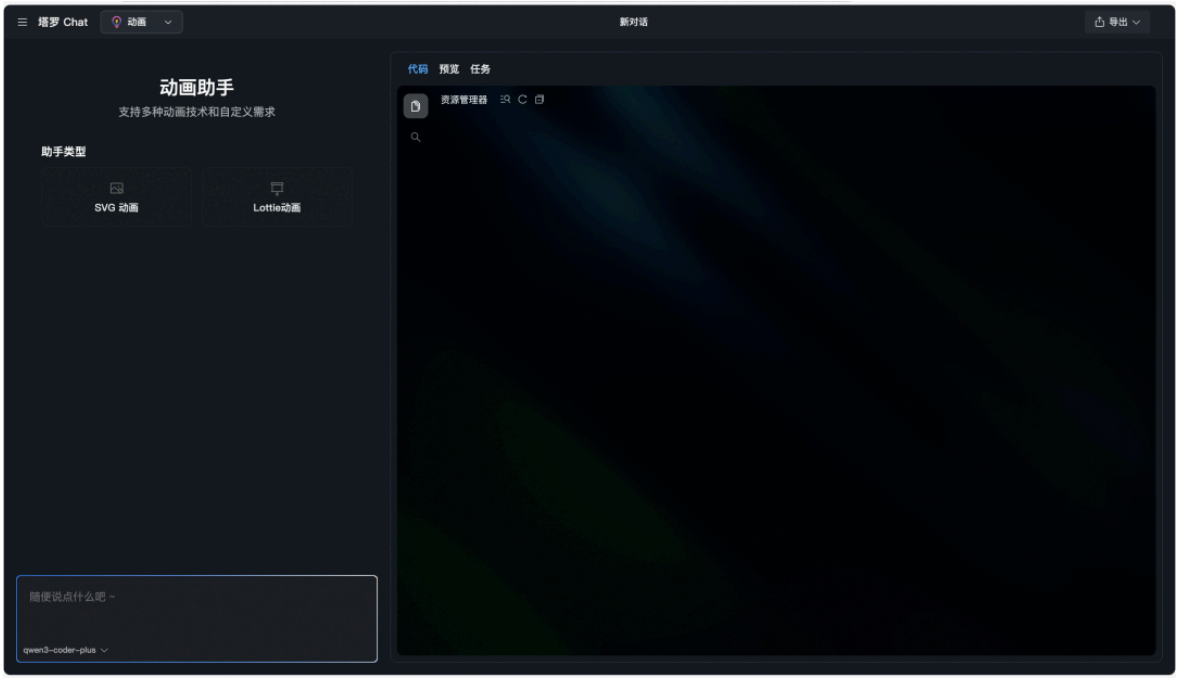
- **对于常规的微动效场景**：比如按钮点击、图标切换、页面入场等，可以通过 Lottie 动画开发助手，**自动解析 Lottie 中隐含的动画语义，精准提取时序、缓动与图层关系，并利用 AI 快速生成可执行且可直接复用的动画代码**，不需要再引入整个 Lottie 库，也不用手动一帧帧还原，既提升了加载性能和开发效率，又方便后期调整和维护。
- **对于复杂的动画场景**：比如 SVG 路径变形、形状渐变，传统实现成本高，且 SVG 本身的坐标系统、路径指令（`d` 属性）、变换矩阵等参数结构复杂，理解与调试成本高， 还极易因精度误差或语法错误导致动画渲染异常。通过 SVG 动画开发助手，**AI 能自动解析 SVG 路径 ，将关键帧串联起来，生成结构清晰、性能良好的 SVG 动画代码**，还能优化关键帧和路径数据，让动画更流畅，代码更简洁。

这两个工具都集成在团队内部的 AI 能力集合（塔罗平台）中，使用者只需要上传文件或输入需求，就能快速得到可用的动画代码，大幅减少重复劳动，把精力集中在交互逻辑和业务实现上。我们希望用 AI 把“做动画”这件事变得更简单、更可靠，真正实现从设计到代码的高效落地。

04 实现方案

塔罗平台支持动画助手

塔罗平台是场景营销前端团队基于前端资产体系发展出的一套内部 AI 工具和服务平台。它的定位是“用 AI 当工具”— 专注于解决编辑器做不了或不好做的复杂交互场景，以及需要快速调用、即时反馈的使用需求。



塔罗支持基于意图的代码生成与 Agentic 模式的任务执行，具备实时效果预览及文件读写、搜索、修改、追问、总结等完整交互能力。

为支持动画助手的实现，需对塔罗进行以下三方面改造：

1. **增强工具化调用能力**：通过构建模块化、可扩展的工具调用机制，实现对特定领域功能的灵活扩展。该架构支持自定义工具接入，为后续功能（如 Lottie 分析工具）提供良好的扩展基础。 基于塔罗现有能力，进一步支持特定场景的代码生成、辅助功能的开发，例如：Lottie 时间轴。



2. **新增 Lottie 动画分析能力：通过工程的方式深度解析 Lottie 文件，提取其中关键动画信息，保证动画分析阶段的100%准确率，再结合 AI 能力**自动生成可直接用于前端的动画代码。具体解析出的内容包括：

- 动画元素的起始与结束时间
- 动画持续时长
- 属性变化类型（如位移、缩放、旋转、透明度等）
- 贝塞尔缓动曲线参数
- 关键帧时间节点

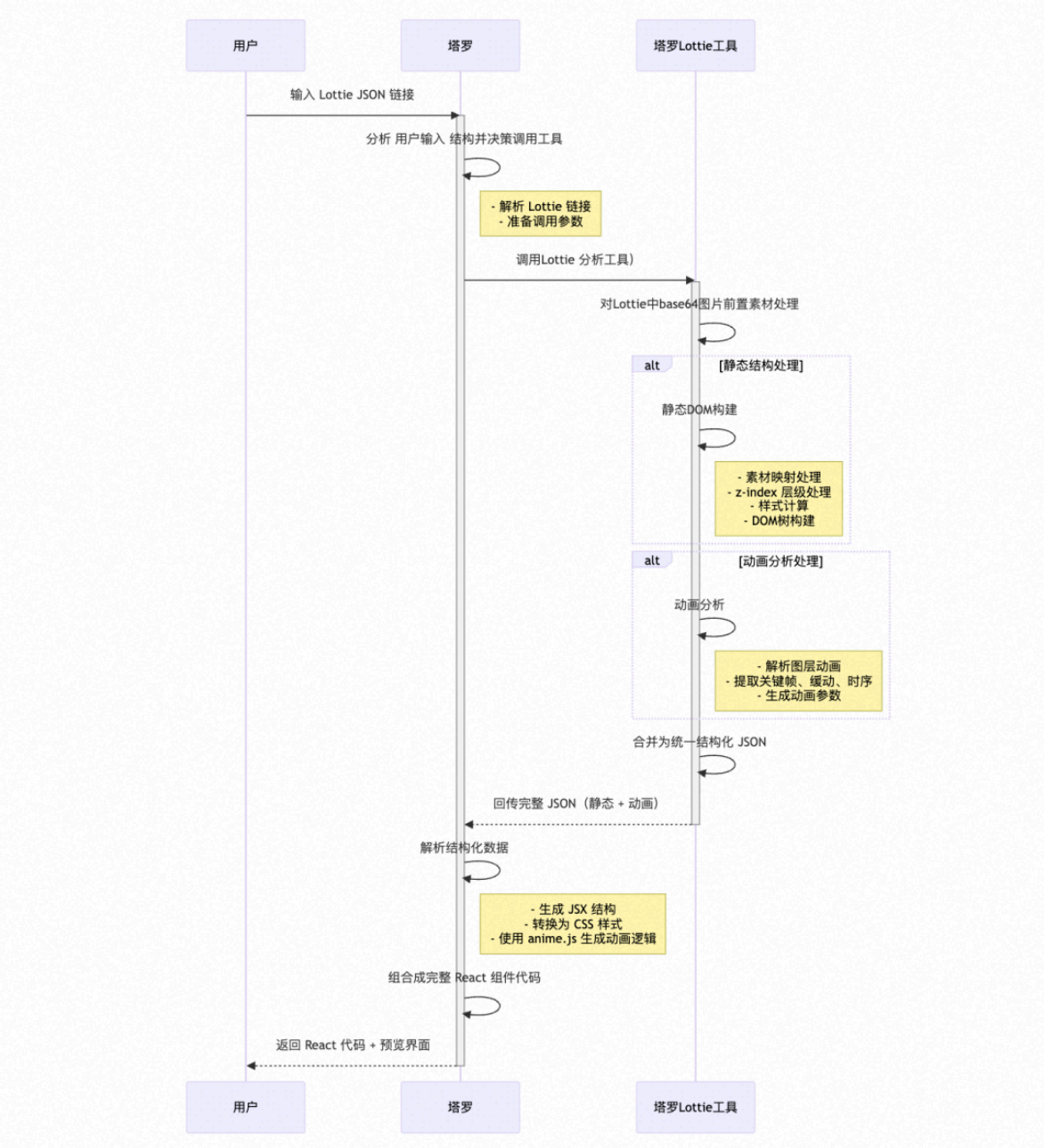
3. 特别针对 SVG 动画的处理预先定义一份“ SVG 帧动画处理指引”（输入/输出、约束、流程、目录、依赖），作为执行规范。仅产出一个 `SVGAnimationPlayer` 组件（默认导出），样式用 CSS Modules，按指引实现“单 SVG 容器 + 帧切换”的渲染逻辑。

Lottie 动画助手

- 核心要求

1. **降低 AI 生成的不确定性**，提高动画实现的准确性以及每次输出的稳定性，所以采用工程的方式前置进行分析
2. 生成的动画代码符合平时开发规范，可直接复用。

- Lottie 工具实现



Lottie Tool 工具定义

- **功能描述：**该工具用于分析 Lottie 动画文件，支持通过 URL 或直接传入 JSON 字符串加载动画数据。工具将解析 Lottie 文件的完整结构，并提取关键动画信息，生成结构化的动画摘要，便于后续代码生成与开发使用。
- **提取内容包括：**基础信息：动画名称、Lottie 版本、画布尺寸（宽度、高度）、帧率（fps）、总帧数、总时长（秒）等；
- **时间轴摘要：**图层（layers）结构、关键属性变化轨迹（如位置、缩放、旋转、透明度等）、关键帧时间点、缓动函数（easing）类型（如线性、贝塞尔曲线等）；
- **资源信息：**内嵌或外部引用的图像资源（如位图、字体等），包括资源 ID、文件名、尺寸、类型及访问 URL；
- **数据源支持：**支持从链接获取 Lottie 文件内容，也支持直接传入原始 JSON 数据。
- **触发条件：**当用户表达“分析 Lottie”、“解析动画文件”、“查看 Lottie 结构”等意图，或提供 Lottie 文件的 URL 链接时，应自动调用此工具。

Lottie Tool 工具执行

这一步主要是对 Lottie 数据进行处理，返回结构化的数据给 AI，**没有把所有工作都交给 AI 是因为纯 AI 难以保证生成结果的准确性，还会带来性能开销、Token 消耗过大以及引入额外的复杂度**，特别是 Lottie 动画的实现和前端 DOM 动画开发的实现方案有一些差异，有许多隐藏的难点例如属性冲突等问题，需要在分析阶段进行规避来保证生成的准确率。

动画分析

处理 Lottie 动画数据，主要目的是分析和归类动画属性，检测循环模式，并按相似性对动画进行分组。

动画段提取（`findAnimationSegments`）

- 从关键帧数组中提取有效的动画段
- 识别缓动类型（Linear、Hold、cubic-bezier）
- 过滤掉值不变的段（优化性能）

循环检测（`detectAndProcessLoops`）

```
// 核心逻辑：寻找重复模式
for (let period = 1; period <= segments.length / 2; period++) {
  if (segments.length % period === 0) {
    // 检查是否为完整的重复循环
  }
}

```

值标准化与缩放

适配塔罗移动端容器展示，移动端容器 `375 * 812` 。

```
const normalizeValueForKey = (jsonString: string, propKey: string) =>
  // 根据属性类型决定是否需要缩放
  switch (propKey) {
    case 'p': // position - 需要缩放
    case 'a': // anchorPoint - 需要缩放
      shouldScaleX = true; shouldScaleY = true;
      break;
    case 's': // scale - 不需要缩放（百分比）
    case 'r': // rotation - 不需要缩放
    // ...
  }
```

```
}

```

动画分组策略

符合开发习惯，不同元素相同变换，统一处理

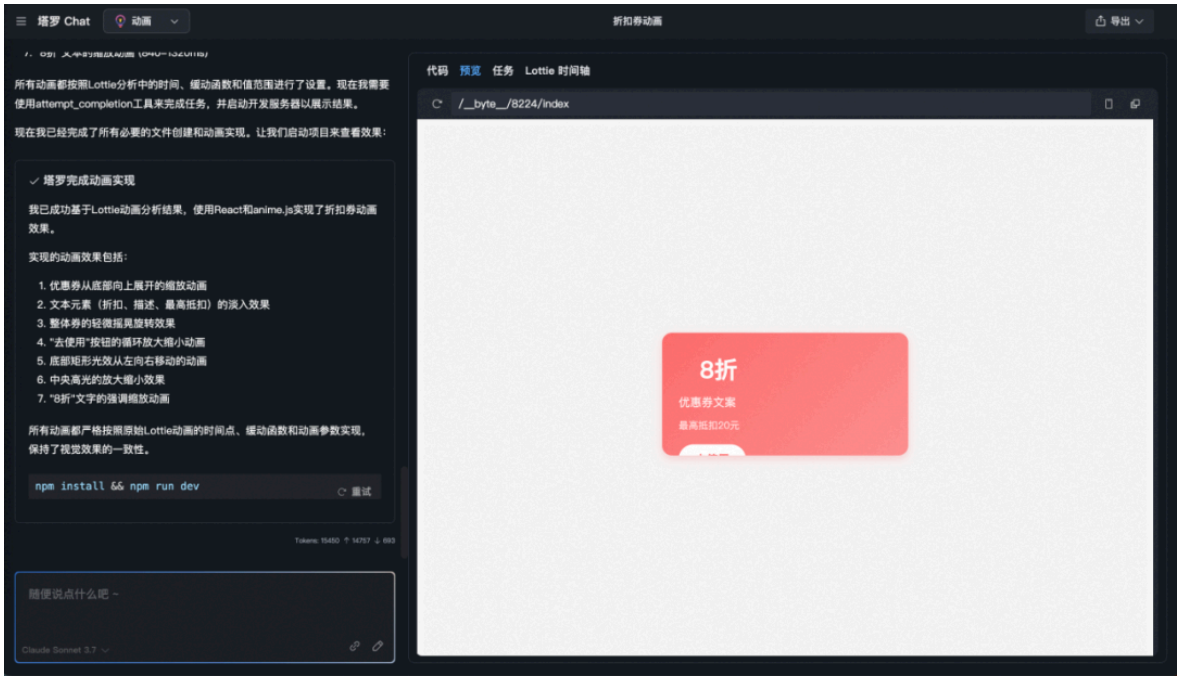
```
const animationKey =
  `prop:${propKey}|isLoop:${isLoop}|loops:${loopCount}|` +
  scaledSegments.map(s => `sf:${s.startFrame},ef:${s.endFrame},...`).jo
```

分组依据：

- 属性类型（position、scale、rotation等）
- 是否为循环动画
- 循环次数
- 具体的动画段参数

静态 dom 还原

为提升动画还原的准确性，我们优化了 AI 的工作模式：不再让其自行构建 DOM 结构，而是由解析系统预先生成与 Lottie 完全一致的标准化 DOM 层级。**AI 在此基础上仅负责动画逻辑实现**，避免了因自由构造结构导致的层级错乱或样式偏差。这一调整使得还原结果更贴近原始设计，便于验证 AI 输出的正确性，也显著提升了动画代码的可靠性和可维护性。



素材映射

素材映射环节旨在对 Lottie 文件中的资源进行前置处理。由于设计师提供的 Lottie 通常包含大量 Base64 编码的内嵌图片，导致文件体积过大，直接影响大模型的输入效率与处理稳定

性。为此，我们通过工程化手段，在解析阶段自动识别 Base64 图片并将其替换为对应的外链，显著减少 Lottie 数据体积。处理后的轻量化结构再交由大模型处理。

```
{
  "assets": [
    {
      "id": "image_0",            // 素材唯一标识
      "w": 82,                    // 原始宽度
      "h": 70,                    // 原始高度
      "u": "images/",             // 路径前缀
      "p": "magnify.png",         // 文件名
      "e": 1                      // 是否嵌入
    }
  ]
}
```



层级处理

确保 DOM 元素的层级与 Lottie 一致：`ind` 越小，层级越高（在上层）；`ind` 越大，越先渲染（在底层）。按此顺序构建 DOM，保证视觉层次正确。

z-Index 计算策略

```
function calculateZIndex(layer, allLayers, baseZIndex = 1000) {
  const maxInd = Math.max(...allLayers.map(l => l.ind || 0));
  const minInd = Math.min(...allLayers.map(l => l.ind || 0));

  // 核心公式：ind越小，z-index越大
  const zIndex = baseZIndex + (maxInd - (layer.ind || 0));

  // 父子关系处理：子元素基础z-index更高
  if (layer.parent !== undefined) {
    return zIndex + 10000;  // 确保子元素在父元素之上
  }

  return zIndex;
}

// 示例：
// ind=1  → z-index=1015 (最前面)
```

```
// ind=7  → z-index=1009 (中间)
// ind=15 → z-index=1001 (最后面)

```

DOM 树构建

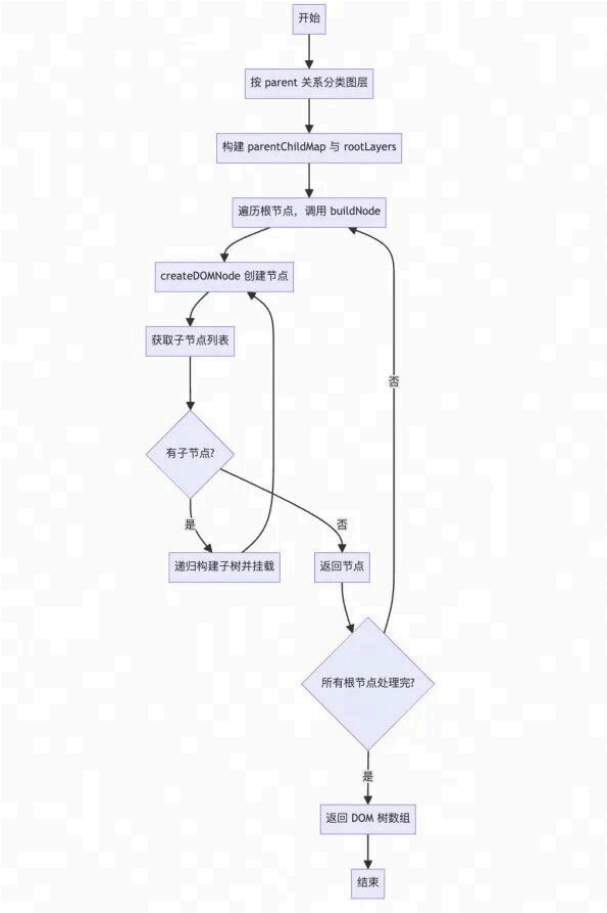
父子关系识别

```
// Lottie中的父子关系通过parent字段定义
{
  "ind": 7,         // 当前图层ID
  "parent": 3,      // 父图层ID（可选）
  "nm": "child"     // 图层名称
}

```

树结构构建算法

递归地为每个图层创建DOM节点，并建立正确的父子关系



样式计算

将 Lottie 的坐标系统精确映射为 CSS 定位，确保元素位置准确。Lottie 基于固定画布（如 750×1624）进行绝对定位，所有图层的坐标由 `ks.p`（position）定义，需按画布尺寸直接转

换为对应的 left 和 top 值，保证布局与原始动画完全一致。

```
// Lottie变换属性
{
  "ks": {
    "p": {"k": [375, 400]},    // position: 锚点在画布中的位置
    "a": {"k": [50, 25]},      // anchor: 元素的锚点（中心点）
    "s": {"k": [100, 100]},    // scale: 缩放比例
    "r": {"k": 0},             // rotation: 旋转角度
    "o": {"k": 100}            // opacity: 透明度
  }
}

```

精确定位算法

Lottie 图层位置与样式到 CSS 的精确映射，核心逻辑如下：

- 坐标转换：基于 Lottie 画布尺寸与目标尺寸的缩放比（scaleX /scaleY），将图层的锚点偏移（position – anchor）转换为 CSS 的 left 和 top 值，确保定位准确；
- 变换处理：将缩放（scale）和旋转（rotation）属性转换为 CSS transform 字符串，仅在值非默认时添加，避免冗余；
- 样式输出：返回包含绝对定位、变换、透明度和 zIndex 的完整样式对象，保证元素在 DOM 中的视觉表现与 Lottie 一致。

```
function calculatePosition(layer, targetWidth, targetHeight, compWidth
  // 1. 提取基础数据
  const position = extractValue(layer.ks?.p?.k, [0, 0]);
  const anchor = extractValue(layer.ks?.a?.k, [0, 0]);
  const scale = extractValue(layer.ks?.s?.k, [100, 100]).map(v => v /
  const rotation = extractValue(layer.ks?.r?.k, 0);
  const opacity = extractValue(layer.ks?.o?.k, 100) / 100;

  // 2. 计算缩放比例
  const scaleX = targetWidth / compWidth;
  const scaleY = targetHeight / compHeight;

  // 3. 核心定位公式
  // 元素左上角位置 = (锚点位置 - 锚点偏移) * 缩放比例
  const finalLeft = (position[0] - anchor[0]) * scaleX;
  const finalTop = (position[1] - anchor[1]) * scaleY;

  // 4. 构建样式
  return {
    position: 'absolute',
```

```
    left: Math.round(finalLeft) + 'px',
    top: Math.round(finalTop) + 'px',
    transform: buildTransform(scale, rotation),
    opacity: opacity,
    zIndex: calculateZIndex(layer)
  };
}

function buildTransform(scale, rotation) {
  const transforms = [ ];

  // 缩放变换
  if (Math.abs(scale[0] - 1) > 0.001 || Math.abs(scale[1] - 1) > 0.001
    transforms.push(`scale(${scale[0]}, ${scale[1]})`);
  }

  // 旋转变换
  if (Math.abs(rotation) > 0.001) {
    transforms.push(`rotate(${rotation}deg)`);
  }

  return transforms.length > 0 ? transforms.join(' ') : 'none';
}

```

转换到CSS

```
.lottie-layer {
  position: absolute;  /* 必须是absolute */
  left: 计算后的x坐标;
  top: 计算后的y坐标;
  transform: scale() rotate();  /* 应用变换 */
  z-index: 根据ind计算的层级;
}

```

AI 生成动画代码

AI 作为代码生成的核心，基于解析阶段输出的结构化数据（domTree、timeline、metadata），结合预设的前端开发规范，自动生成高还原度的动画组件代码。将单个动画的开发耗时从小时级缩短至分钟级。

```
{
```

```
  timeline: {
    // 按图层和属性组织的动画数据
    "layer_1_position": {
      segments: [...],
      isLoop: false,
      loopCount: 1
    }
  },
  domTree: {
    // 完整的 DOM 结构描述
    type: "div",
    props: { className: "container" },
    children: [...]
  },
}

```

基于 domTree 构建 DOM 结构

- 严格按照 domTree 构建：不允许任何结构调整
- 精确定位系统：domTree 已包含完整定位信息
- 资源映射：图像资源转换为对应 DOM 元素

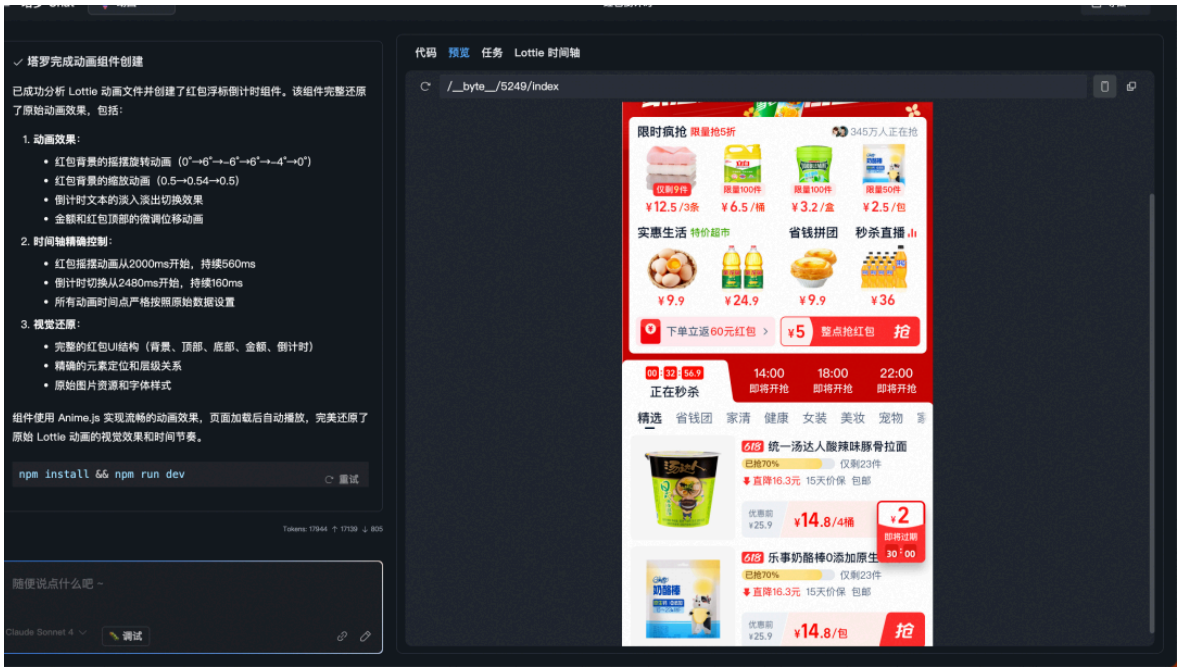
基于 timeline 生成 anime.js 动画

将 `timeline` 中的关键帧、持续时间、延迟、缓动曲线等信息，转化为 anime.js 的配置对象，确保动画时序与视觉表现一致。

```
timeline 数据 → anime.js 配置
├── segments 数组 → keyframes 数组
├── easing 参数 → anime.js easing 格式
├── 时间计算 → duration 和 delay
└── 循环检测 → loop 参数
```

输出为完整的 React 组件

生成完整的可预览的 React 组件代码结合塔罗预览能力进行预览



SVG 动画助手

- 核心作用

一句话总结：将“开发者看不懂”的 SVG 路径序列，转化为结构化、可读、可维护、高性能的可执行动画逻辑。

AI 的作用

- **语义理解：**从无意义的 `d` 属性中识别形状语义（如“心形”“波浪”“箭头”），理解“这是什么动画”
- **逻辑串联：**分析多个 SVG 片段的形态变化，自动构建“起始→过渡→结束”的动画流程
- **数据优化：**自动简化冗余路径点、修复精度误差、压缩数据体积。

- 实现方案

核心思路

预先定义一份“SVG 帧动画处理指引”，作为 AI 执行的标准化规范。该指引明确输入输出格式、环境约束、实现流程、目录结构与依赖管理，确保在 Byte 环境下稳定、一致地生成可运行的 SVG 动画组件。

组件产物

仅产出一个 `SVGAnimationPlayer` 组件（默认导出），样式用 CSS Modules，按指引实现“单 SVG 容器 + 帧切换”的渲染逻辑。

```
- **项目结构**:
  ${workDir}
  ├── package.json                    # 项目依赖管理文件
  ├── src/
  │   ├── index.tsx                   # 应用入口文件，假设已存在byte系统中，不用创建
  │   ├── components/                 #
  │   │   ├── SVGAnimationPlayer/                 # 动画播放器组件
  │   │   │   ├── pathData.ts                  # 存放svg帧数据
  │   │   ├── AnimationControls/                  # 动画控制器组件，假设已存在byte系统中，不用
  创建      You, 上周 • fix: 增加教程页
```

数据约定

- 帧数据抽离：
  - 所有 SVG 帧中变化的 `pathData.ts` 属性统一提取至 `/src/components/SVGAnimationPlayer/pathData.ts`。
  - 共同的 `viewBox`、`width`、`height`、`fill`、`stroke` 等属性保留在组件内静态定义，提升渲染一致性。

```ts
// /src/components/SVGAnimationPlayer/pathData.ts
export const pathFrames: string[] = [
  "M10 10 H 90 V 90 H 10 Z",
  "M20 20 H 80 V 80 H 20 Z",
  // ...更多帧
];
```

动画控制集成

预先定义 `AnimationControls`，通过 Byte 虚拟文件能力，支持 AI 无需重复生成动画控制模块，直接复用 AnimationControls 组件，通过 props 连接播放/暂停、前后帧、跳转、重播、倍速等控制，动画组件只负责状态与渲染，实现每次输入都会有相同的控制能力。



05 效果演示

通过该方案，可在几分钟内自动生成完整的动画相关代码，显著减少与设计沟通动画细节的成本，避免手动实现和反复调试动画所带来的耗时问题。

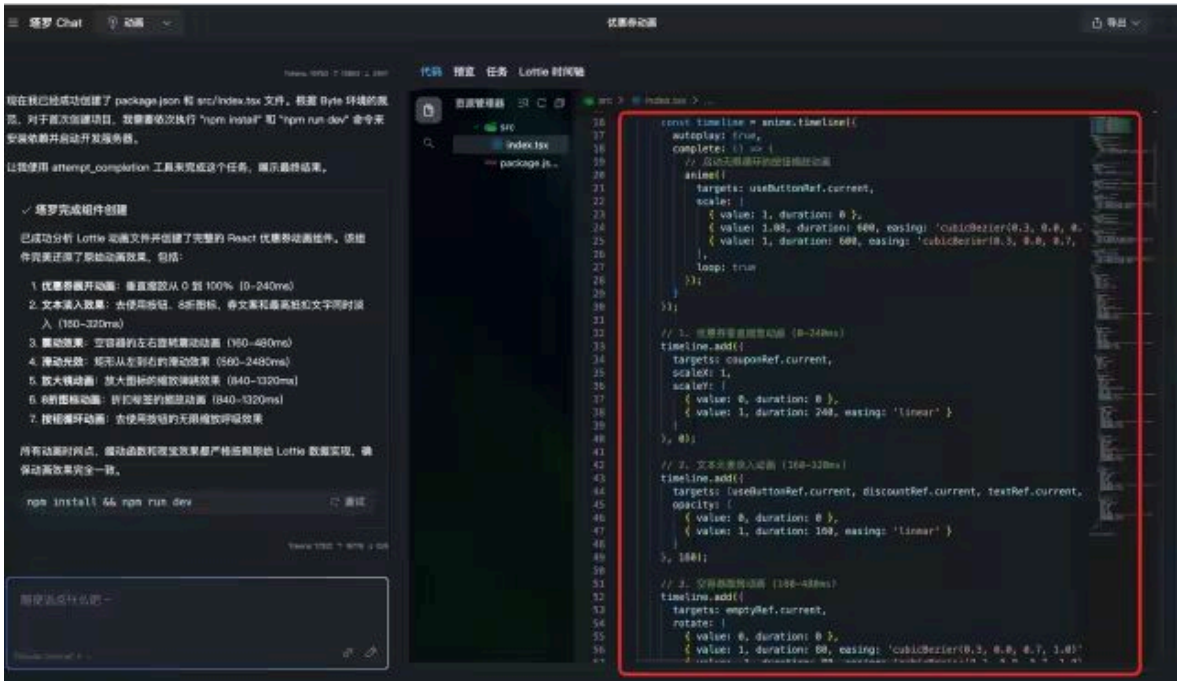
以往需数小时完成（视动画复杂程度而定）的动画开发流程（包括动画分析、编码实现与调试），**可压缩至几分钟内自动完成**。可以大幅提升开发效率，也显著提高了动画实现的准确性和一致性，确保动效还原度与设计高度匹配。

Lottie 动画助手

- 使用方法

输入 lottie 文件的 url 链接。

1.生成所见即所得的动画代码，可直接在 Cursor 编辑器里应用。



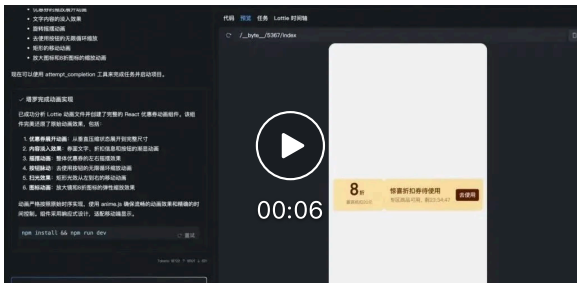
2.提供可视化的 Lottie 时间轴以及对应单节点的 animeJs 和 css 代码生成。

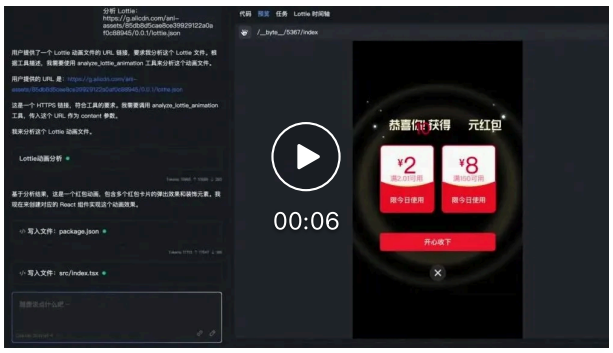
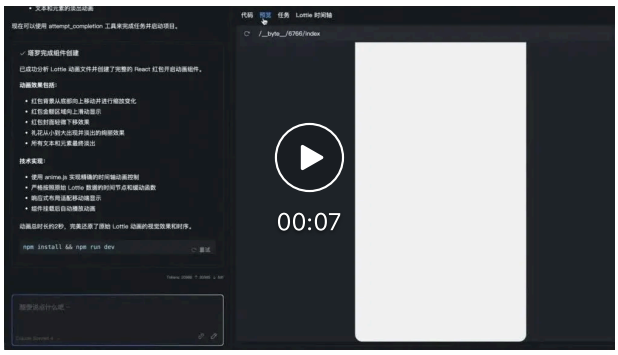
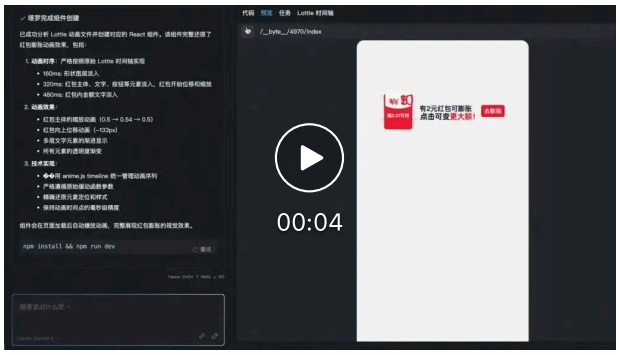
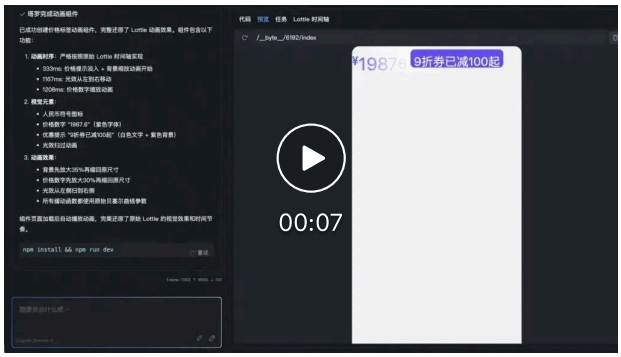




- 测试案例

以下是我们提供的测试案例，用于验证动画解析与生成能力的边界情况，仅供参考。

Lottie 地址	实际效果	评测
优惠券动画		✅ 动画还原 ✅ 时间轴 ✅ ui 还原 100%

		
红包浮标倒计时		✅ 动画还原 ✅ 时间轴 ✅ ui 还原 100%
红包pop		✅ 动画还原 ✅ 时间轴 ✅ ui还原度 95%
红包膨胀		✅ 动画还原 ✅ 时间轴 ✅ ui还原度 95%
浮层红包膨胀		✅ 动画还原 ✅ 时间轴 ✅ ui 还原度 80%
价格动画 （uni 平台随机找的）		✅ 扫光动画还原 ✅ 价格缩放还原 ✅ tip 缩放动画 ✅ 时间轴 ✅ ui 还原度 90%

SVG 动画助手

输入基于 sketch 复制的 svg 片段代码。

提供播放/暂停、前后帧、跳转、重播、倍速等能力可对生成的 svg 动画进行调试。



06 团队介绍

本文作者香芋，来自淘天集团-用户场景营销技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2870

留言

写留言

还没有留言，来写首评